■■ Master C++ & DSA Learning Dashboard

A Production-Grade Roadmap & Interactive Milestone Tracker from Language Fundamentals to Advanced Algorithmic Mastery

---

■ Dashboard Overview & Metrics

Metric	Target / Detail	Status
Total Modules	12 Core Modules (1 C++ Revision + 11 DSA Domains)	■ Active
Theoretical Subtopics	85+ Concepts (Memory, Complexity, Patterns, Algorithms)	■ Planned
Must-Do Famous Problems	105 Curated Interview Problems (LeetCode / Striver A2Z)	■ 0 / 105 Completed
Estimated Duration	16 Weeks (Dedicated Systematic Study)	■■ In Progress
Offline Version	PDF Copy Available (DSA_Syllabus.pdf)	■ Compiled

---

■ Milestone & Checkpoint Tracker

Use this checklist as your high-level progress dashboard. Check off each milestone as you master the underlying theory and solve the associated must-do problems.

■ Phase 1: C++ Language & Memory Mastery (Weeks 1–2)

- ❏ **CP-01:** Basic C++ syntax, fast I/O (**`cin.tie`**), primitive data types, & range overflow limits.
- ❏ **CP-02:** Pass-by-value vs pass-by-reference (**`&`**), pointer arithmetic, & memory addresses (**`*`**, **`&`**).
- ❏ **CP-03:** Sequence STL containers (**`vector`**, **`deque`**, **`list`**) & capacity management (**`reserve`**).
- ❏ **CP-04:** Associative STL containers (**`set`**, **`map`**, **`unordered_set`**, **`unordered_map`**) & hash tables.
- ❏ **CP-05:** Container adaptors (**`stack`**, **`queue`**, **`priority_queue`** Min/Max heaps) & essential STL algorithms.
- ❏ **CP-06:** Advanced C++: Smart pointers (**`unique_ptr`**, **`shared_ptr`**), move semantics (**`&&`**), & lambda comparators.

■ Phase 2: Core Linear Data Structures & Search Space (Weeks 3–6)

❏ **CP-07:** Array pattern mastery: Two Pointers, Sliding Window (Fixed & Variable), Prefix Sum, & Kadane's algorithm.

❏ **CP-08:** Linked List pointer manipulation: In-place reversal, fast & slow pointers, dummy nodes, & LRU cache.

❏ **CP-09:** Monotonic Stack & Queue patterns (Next Greater Element, Sliding Window Maximum, Largest Histogram).

❏ **CP-10:** Binary Search fundamentals: Lower/Upper bounds, Rotated Array search, & Binary Search on Answer Space.

■ Phase 3: Non-Linear Structures, Recursion & Trees (Weeks 7–10)

❏ **CP-11:** Recursive state-space tree search, inclusion/exclusion pattern, & backtracking with pruning.

❏ **CP-12:** Binary Tree traversals (DFS Pre/In/Post, BFS Level Order) & Lowest Common Ancestor (LCA).

❏ **CP-13:** Binary Search Tree invariants, BST operations (Insert, Search, Delete), & Inorder traversal property.

❏ **CP-14:** Binary Heaps (*O(N)* Build-Heap), Top-K elements pattern, K-Way Merging, & Two-Heaps Median tracking.

■ Phase 4: Graphs, Dynamic Programming & Advanced DSA (Weeks 11–16)

❏ **CP-15:** Bit manipulation formulas (XOR tricks, `x & (x-1)`, power of 2) & Bitmasking state compression.

❏ **CP-16:** Graph traversals (BFS/DFS), Topological Sort (Kahn's algorithm), & Disjoint Set Union (DSU).

❏ **CP-17:** Graph shortest paths (Dijkstra, Bellman-Ford, Floyd-Warshall) & Minimum Spanning Trees (Kruskal/Prim).

❏ **CP-18:** Dynamic Programming foundations: 1D DP, Grid DP, 0/1 & Unbounded Knapsack patterns.

❏ **CP-19:** Advanced DP: Longest Common Subsequence (LCS), LIS (*O(N \log N)* BS optimization), & Matrix Chain DP.

❏ **CP-20:** Advanced structures: Trie (Prefix Tree), Segment Tree range queries, Fenwick Tree (BIT), & KMP string match.

■ Module Navigation Index

■ Module 0: C++ Quick Revision & Mastery

0.1 Basic C++

- **Syntax & I/O:**
  - Structure of a C++ program (`#include <iostream>`, `main()`).
  - Fast I/O trick for competitive programming/DSA:

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
```

- **Data Types, Ranges & Precision:** Primitive types (`int`, `long long`, `float`, `double`, `char`, `bool`), overflow/underflow boundaries (*2^{31}-1* vs *2^{63}-1*).
- **Control Flow:** Conditionals (`if`, `else if`, `switch`), Loops (`for`, `while`, `do-while`, range-based `for`).
- **Arrays & Strings:** Fixed arrays vs `std::string` manipulation (`length()`, `substr()`, `find()`, `append()`).
- **Functions & Parameter Passing:** Pass-by-Value vs Pass-by-Reference (`&`), Pass-by-Const-Reference (`const std::string& str`).
- **Pointers & Memory Addresses:** Address-of (`&`), Dereference (`*`), Pointer arithmetic, Null pointers (`nullptr`).

0.2 Intermediate C++ & Standard Template Library (STL)

- **Sequence Containers:**
  - `std::vector`: Dynamic array, amortized *O(1)* push_back, capacity vs size, `reserve()`, `emplace_back()`.
  - `std::deque`: Double-ended queue, *O(1)* push/pop at both ends.
  - `std::list`: Doubly linked list, *O(1)* insertions/deletions anywhere given an iterator.
- **Associative Containers:**
  - `std::set` / `std::map`: Self-balancing Red-Black Tree, sorted keys, *O(\log N)* operations.
  - `std::unordered_set` / `std::unordered_map`: Hash table, average *O(1)* operations, worst-case *O(N)*.
  - `std::pair` & `std::tuple`: Multi-value binding.
- **Container Adaptors:**
  - `std::stack`: LIFO operations (`push`, `pop`, `top`).
  - `std::queue`: FIFO operations (`push`, `pop`, `front`).
  - `std::priority_queue`: Max-heap by default (`top`, `push`, `pop`), Min-heap via `greater<int>`.
- **Essential STL Algorithms:** `std::sort`, `std::binary_search`, `std::lower_bound`, `std::upper_bound`, `std::next_permutation`, `std::accumulate`.

- **Dynamic Memory & OOP:** Dynamic allocation (`new`/`delete`), Classes vs Structs, Constructors, Destructors, Polymorphism.

0.3 Advanced C++

- **Smart Pointers (RAII):** `std::unique_ptr` (exclusive ownership), `std::shared_ptr` (reference-counted), `std::weak_ptr` (cyclic protection).
- **Move Semantics & Rvalue References:** Rvalue reference (`T&&`), `std::move`, move constructors, avoiding deep copies.
- **Templates & Generic Programming:** Function templates, Class templates, Template specialization.
- **Lambda Expressions & Custom Comparators:**
  - Sorting comparators: `sort(vec.begin(), vec.end(), { return a.x < b.x; });`
  - Custom priority queue comparators.
- **Custom Hash Functions:** Custom hashes for `std::unordered_map<pair<int,int>, int>` using XOR / `boost::hash_combine`.
- **Memory Alignment & Cache Locality:** Spatial & Temporal locality in array traversals vs node-based data structures.

■ Module 1: Arrays & Strings

1.1 Core Concepts & Theory

- Memory layout of 1D and 2D arrays (Row-Major vs Column-Major ordering).
- Time and Space complexity of Array operations (Access *O(1)*, Search *O(N)*, Insertion/Deletion *O(N)*).
- String immutability vs mutability concepts, ASCII & UTF-8 character encodings.

1.2 Theoretical Subtopics & Algorithmic Patterns

- **Two Pointers Technique:** Opposite direction (palindrome check, 2-sum) and Same direction (fast/slow, duplicate removal).
- **Sliding Window:** Fixed-size window (max sum of subarray K) and Variable-size window (longest substring with K distinct chars).
- **Prefix Sum & Difference Array:** *O(1)* range sum queries, *O(1)* range updates using difference array.
- **Kadane's Algorithm:** Maximum subarray sum intuition, local vs global optimal state transition.
- **Dutch National Flag Algorithm:** 3-way partitioning algorithm (*O(N)* time, *O(1)* space).
- **2D Matrix Manipulation:** In-place matrix rotation, spiral traversal, matrix multiplication.

1.3 Must-Do Famous Problems

- **Two Sum** *(Array, Hash Table)* — LeetCode #1
- **Best Time to Buy and Sell Stock** *(Array, Dynamic Programming)* — LeetCode #121
- **Maximum Subarray (Kadane's Algorithm)** *(Array, DP)* — LeetCode #53
- **3Sum** *(Two Pointers, Array, Sorting)* — LeetCode #15

- **Container With Most Water** *(Two Pointers, Greedy)* — LeetCode #11
- **Trapping Rain Water** *(Two Pointers, Stack, Array)* — LeetCode #42
- **Find All Anagrams in a String** *(Sliding Window, Hash Table)* — LeetCode #438
- **Subarray Sum Equals K** *(Prefix Sum, Hash Table)* — LeetCode #560
- **Spiral Matrix** *(2D Array, Simulation)* — LeetCode #54
- **Minimum Window Substring** *(Sliding Window, Hash Table)* — LeetCode #76

■ Module 2: Linked Lists

2.1 Core Concepts & Theory

- Linked List Node structure (Data field, Next/Prev pointer fields).
- Memory allocation (Non-contiguous node distribution vs Array contiguous memory).
- Variations: Singly Linked List, Doubly Linked List, Circular Linked List.

2.2 Theoretical Subtopics & Algorithmic Patterns

- **Pointer Manipulation:** In-place node insertion, deletion, and pointer updating without loss of references.
- **Fast & Slow Pointers (Floyd's Cycle Detection):** Mathematical proof of cycle detection ($2k - k = n \cdot C$), finding cycle entry node.
- **Linked List Reversal:** Iterative 3-pointer method (`prev`, `curr`, `next`) and Recursive reversal.
- **Dummy Node Pattern:** Eliminating edge cases for head updates.
- **Merge Sort on Linked Lists:** Finding middle using slow/fast pointers, dividing list, merging sorted lists ($O(N \log N)$ time, $O(1)$ space).
- **Cache Eviction Architectures:** Combined Hash Map + Doubly Linked List for LRU Cache ($O(1)$ get and put).

2.3 Must-Do Famous Problems

- **Reverse Linked List** *(Iterative & Recursive)* — LeetCode #206
- **Middle of the Linked List** *(Fast & Slow Pointers)* — LeetCode #876
- **Linked List Cycle I & II** *(Floyd's Cycle Detection)* — LeetCode #141 / #142
- **Merge Two Sorted Lists** *(Two Pointers, Recursion)* — LeetCode #21
- **Remove Nth Node From End of List** *(Two Pointers)* — LeetCode #19
- **Reorder List** *(Middle split + Reverse + Merge)* — LeetCode #143
- **Add Two Numbers** *(Math, Linked List)* — LeetCode #2
- **Merge k Sorted Lists** *(Heap / Divide & Conquer)* — LeetCode #23
- **LRU Cache** *(Hash Table, Doubly Linked List)* — LeetCode #146
- **LFU Cache** *(Hash Table, Doubly Linked List, Frequency Map)* — LeetCode #460

■ Module 3: Stacks & Queues

3.1 Core Concepts & Theory

- **Stack:** LIFO (Last In First Out) paradigm, Function Call Stack execution model.
- **Queue:** FIFO (First In First Out) paradigm, Task scheduling queues.
- Array-based vs Linked List-based implementations, Array overflow/underflow handling.

3.2 Theoretical Subtopics & Algorithmic Patterns

- **Monotonic Stack:** Next Greater Element (NGE), Next Smaller Element (NSE), Previous Greater/Smaller Element.
- **Monotonic Queue:** Sliding Window Maximum tracking using Deque (*O(N)* amortized time).
- **Expression Evaluation & Parsing:** Infix to Postfix (Shunting-yard Algorithm), Postfix Evaluation using Stack.
- **Min Stack / Max Stack:** Auxiliary stack technique vs *O(1)* space math encoding (`2 * val - minVal`).

3.3 Must-Do Famous Problems

- **Valid Parentheses** *(Stack)* — LeetCode #20
- **Min Stack** *(Design, Stack)* — LeetCode #155
- **Evaluate Reverse Polish Notation** *(Stack, Math)* — LeetCode #150
- **Daily Temperatures** *(Monotonic Stack)* — LeetCode #739
- **Next Greater Element I & II** *(Monotonic Stack)* — LeetCode #496 / #503
- **Largest Rectangle in Histogram** *(Monotonic Stack)* — LeetCode #84
- **Sliding Window Maximum** *(Monotonic Queue, Deque)* — LeetCode #239
- **Online Stock Span** *(Monotonic Stack)* — LeetCode #901
- **Implement Queue using Stacks** *(Design, Stack)* — LeetCode #232
- **Basic Calculator I & II** *(Stack, Math, String Parsing)* — LeetCode #224 / #227

■ Module 4: Recursion & Backtracking

4.1 Core Concepts & Theory

- Recursion tree visualization, Base Case, Recursive State Transition, Call Stack Depth & Stack Overflow.
- Backtracking paradigm: Explore, Choose, Un-choose (State Reset).

4.2 Theoretical Subtopics & Algorithmic Patterns

- **State Space Tree Search:** Systematic exploration of search space.
- **Subsets / Power Set (Include/Exclude Pattern):** Decision tree branches for each element.
- **Combinations & Permutations:** Handling duplicate elements via sorting and frequency maps / boolean visited arrays.

- **Pruning:** Early termination of invalid recursive paths to avoid TLE.
- **Constraint Satisfaction Problems:** Grid backtracking, N-Queens placement validation, Sudoku cell constraints.

4.3 Must-Do Famous Problems

- **Subsets I & II** *(Backtracking, Bit Manipulation)* — LeetCode #78 / #90
- **Permutations I & II** *(Backtracking)* — LeetCode #46 / #47
- **Combination Sum I & II** *(Backtracking)* — LeetCode #39 / #40
- **Letter Combinations of a Phone Number** *(Backtracking, String)* — LeetCode #17
- **Word Search** *(Matrix Backtracking, DFS)* — LeetCode #79
- **Palindrome Partitioning** *(Backtracking, DP)* — LeetCode #131
- **N-Queens** *(Backtracking, Constraint Satisfaction)* — LeetCode #51
- **Sudoku Solver** *(Backtracking, Constraint Satisfaction)* — LeetCode #37

■ Module 5: Sorting & Binary Search

5.1 Core Concepts & Theory

- **Sorting Categories:** Comparison-based vs Non-comparison-based, In-place vs Out-of-place, Stable vs Unstable sorts.
- **Binary Search Paradigm:** Reducing search space by half at each step (*O(\log N)* time), Condition for Monotonicity.

5.2 Theoretical Subtopics & Algorithmic Patterns

- **Sorting Algorithms Breakdown:**
  - Insertion Sort, Selection Sort, Bubble Sort (*O(N^2)*).
  - Merge Sort (*O(N \log N)* Stable), Quick Sort (*O(N \log N)* Average, Pivot Selection strategies).
  - QuickSelect Algorithm (*O(N)* average time for Kth element).
  - Counting Sort, Radix Sort (*O(N + K)* non-comparison).
- **Binary Search Variants:**
  - Standard BS, Order-Agnostic BS.
  - Lower Bound (`first element >= target`), Upper Bound (`first element > target`).
  - Binary search on rotated sorted arrays (identifying sorted half).
- **Binary Search on Answer Space:** Monotonic predicate functions *f(x) \to \{\text{True, False}\}*, defining search range `[low, high]`.

5.3 Must-Do Famous Problems

- **Binary Search** *(Search, Array)* — LeetCode #704
- **Search in Rotated Sorted Array I & II** *(Binary Search)* — LeetCode #33 / #81

- **Find First and Last Position of Element in Sorted Array** *(Lower/Upper Bound)* — LeetCode #34
- **Kth Largest Element in an Array** *(QuickSelect / Heap)* — LeetCode #215
- **Search a 2D Matrix I & II** *(Binary Search)* — LeetCode #74 / #240
- **Find Minimum in Rotated Sorted Array** *(Binary Search)* — LeetCode #154
- **Koko Eating Bananas** *(Binary Search on Answer)* — LeetCode #875
- **Capacity To Ship Packages Within D Days** *(Binary Search on Answer)* — LeetCode #1011
- **Aggressive Cows / Book Allocation Problem** *(Binary Search on Answer)* — Standard Pattern
- **Median of Two Sorted Arrays** *(Advanced Binary Search)* — LeetCode #4

■ Module 6: Trees & Binary Search Trees (BST)

6.1 Core Concepts & Theory

- Tree Definitions: Root, Child, Parent, Leaf, Depth, Height, Ancestor, Subtree.
- Binary Tree Types: Full, Complete, Perfect, Balanced, Degenerate.
- Binary Search Tree Invariant: $\text{Left Subtree} < \text{Node} < \text{Right Subtree}$.

6.2 Theoretical Subtopics & Algorithmic Patterns

- **Tree Traversals:**
  - Depth-First: Inorder (Left-Node-Right), Preorder (Node-Left-Right), Postorder (Left-Right-Node) via Recursion & Iterative Stacks.
  - Breadth-First: Level-Order Traversal using Queue.
- **Lowest Common Ancestor (LCA):** Top-down vs Bottom-up search logic for BT and BST.
- **Tree Construction:** Reconstructing tree from (Preorder + Inorder) or (Postorder + Inorder).
- **Diameter & Path Sums:** Global maximum update during postorder traversal.
- **BST Operations:** Search, Insert, Delete node (3 cases: Leaf, 1 Child, 2 Children via Inorder Successor/Predecessor).
- **Self-Balancing Concepts:** AVL Rotations (LL, RR, LR, RL), Red-Black tree properties (conceptual).

6.3 Must-Do Famous Problems

- **Maximum Depth of Binary Tree** *(DFS/BFS)* — LeetCode #104
- **Invert / Flip Binary Tree** *(Recursion)* — LeetCode #226
- **Diameter of Binary Tree** *(DFS, Global Max)* — LeetCode #543
- **Binary Tree Level Order Traversal** *(BFS, Queue)* — LeetCode #102
- **Lowest Common Ancestor of a Binary Tree** *(Recursion)* — LeetCode #236
- **Validate Binary Search Tree** *(BST Invariant, Range Check)* — LeetCode #98
- **Kth Smallest Element in a BST** *(Inorder Traversal)* — LeetCode #230
- **Construct Binary Tree from Preorder and Inorder Traversal** *(Recursion, Hash Table)* — LeetCode #105

- **Binary Tree Maximum Path Sum** *(Postorder DFS, Dynamic Programming)* — LeetCode #124
- **Serialize and Deserialize Binary Tree** *(Design, BFS/DFS)* — LeetCode #297

■■ Module 7: Heaps & Priority Queues

7.1 Core Concepts & Theory

- Complete Binary Tree representation in Array (Parent at `i/2`, Left Child at `2i`, Right Child at `2i+1`).
- Max-Heap vs Min-Heap properties.
- Building Heap (`Heapify`): *O(N)* time complexity proof vs *O(N \log N)* successive insertion.

7.2 Theoretical Subtopics & Algorithmic Patterns

- **Heap Operations:** `push` (*O(\log N)*), `pop` (*O(\log N)*), `top` (*O(1)*).
- **Top-K Elements Pattern:** Maintaining a Min-Heap of size K for K largest elements.
- **K-Way Merge Pattern:** Merging K sorted streams using Priority Queue.
- **Two-Heaps Pattern:** Min-Heap + Max-Heap combination to track dynamic median in *O(1)* time.

7.3 Must-Do Famous Problems

- **Kth Largest Element in a Stream** *(Min-Heap)* — LeetCode #703
- **Top K Frequent Elements** *(Min-Heap / Bucket Sort)* — LeetCode #347
- **K Closest Points to Origin** *(Max-Heap / QuickSelect)* — LeetCode #973
- **Task Scheduler** *(Max-Heap, Greedy, Queue)* — LeetCode #621
- **Reorganize String** *(Max-Heap, Greedy)* — LeetCode #767
- **Find Median from Data Stream** *(Two Heaps: Min & Max)* — LeetCode #295
- **Merge k Sorted Lists** *(Priority Queue / Divide & Conquer)* — LeetCode #23
- **Smallest Range Covering Elements from K Lists** *(Min-Heap, Sliding Window)* — LeetCode #632

■ Module 8: Hashing & Bit Manipulation

8.1 Core Concepts & Theory

- **Hashing:** Hash Functions, Load Factor (*\alpha = N/M*), Collision Resolution (Separate Chaining vs Open Addressing: Linear/Quadratic Probing, Double Hashing).
- **Bit Manipulation:** Binary representations, Two's Complement, Bitwise operators (`&`, `|`, `^`, `~`, `<<`, `>>`).

8.2 Theoretical Subtopics & Algorithmic Patterns

- **Bit Manipulation Core Formulas:**
  - Check *i*-th bit set: `(n & (1 << i)) != 0`
  - Set *i*-th bit: `n | (1 << i)`

- Clear *i*-th bit: `n & ~(1 << i)`
- Toggle *i*-th bit: `n ^ (1 << i)`
- Unset lowest set bit: `n & (n - 1)` (Kernighan's Algorithm)
- Isolate lowest set bit: `n & (-n)`
- Check Power of 2: `(n > 0) && ((n & (n - 1)) == 0)`

- **Bitmasking:** Representing sets as integers for state compression (*2^N* states).

8.3 Must-Do Famous Problems

- **Single Number I, II, & III** *(XOR Bitwise Logic)* — LeetCode #136 / #137 / #260
- **Number of 1 Bits (Hamming Weight)** *(Bit Operations)* — LeetCode #191
- **Counting Bits** *(Bitwise DP)* — LeetCode #338
- **Reverse Bits** *(Bit Manipulation)* — LeetCode #190
- **Group Anagrams** *(Hash Map, Categorization)* — LeetCode #49
- **Longest Consecutive Sequence** *(Unordered Set, O(N) Search)* — LeetCode #128
- **Subarray Sum Equals K** *(Prefix Sum + Hash Map)* — LeetCode #560
- **Bitwise AND of Numbers Range** *(Bit Shift Logic)* — LeetCode #201
- **Subsets using Bitmasking** *(Combinatorics, Bitmask)* — LeetCode #78

■ Module 9: Graphs

9.1 Core Concepts & Theory

- Graph Representations: Adjacency Matrix (*O(V^2)* space) vs Adjacency List (*O(V + E)* space).
- Terminology: Directed, Undirected, Weighted, Unweighted, Cyclic, Acyclic (DAG), Connected Components, Bipartite.

9.2 Theoretical Subtopics & Algorithmic Patterns

- **Graph Traversals:**
  - **BFS:** Shortest path in unweighted graphs using Queue (*O(V + E)*).
  - **DFS:** Recursive exploration using Call Stack (*O(V + E)*).
- **Cycle Detection:**
  - Undirected Graph: BFS/DFS (visited parent check) or DSU.
  - Directed Graph: DFS (In-degree / Recursion Stack tracking) or Kahn's Algorithm.
- **Topological Sort (DAG):**
  - Kahn's Algorithm (BFS with In-degree array).
  - DFS with Stack.
- **Disjoint Set Union (DSU / Union-Find):** Path Compression & Union by Rank/Size (*O(\alpha(N))* amortized time per operation).

- **Shortest Path Algorithms:**
  - **Dijkstra's Algorithm:** Non-negative weights using Priority Queue ($O((V + E) \log V)$).
  - **Bellman-Ford Algorithm:** Single-source shortest paths with negative weights & negative cycle detection ($O(V \cdot E)$).
  - **Floyd-Warshall Algorithm:** All-pairs shortest paths using 3D/2D DP ($O(V^3)$).
- **Minimum Spanning Tree (MST):**
  - **Kruskal's Algorithm:** Greedy Edge sorting + DSU ($O(E \log E)$).
  - **Prim's Algorithm:** Greedy Vertex expansion + Priority Queue ($O(E \log V)$).
- **Advanced Graph Theory:** Tarjan's Algorithm for Bridges & Articulation Points (Low-link values).

9.3 Must-Do Famous Problems

- **Number of Islands** *(Grid BFS/DFS, Connected Components)* — LeetCode #200
- **Clone Graph** *(Graph Hash Map, BFS/DFS)* — LeetCode #133
- **Course Schedule I & II** *(Topological Sort, Cycle Detection)* — LeetCode #207 / #210
- **Is Graph Bipartite?** *(Coloring BFS/DFS)* — LeetCode #785
- **Word Ladder** *(Shortest Path BFS)* — LeetCode #127
- **Network Delay Time** *(Dijkstra's Algorithm)* — LeetCode #743
- **Cheapest Flights Within K Stops** *(Bellman-Ford / Modified Dijkstra)* — LeetCode #787
- **Redundant Connection** *(DSU / Cycle Detection)* — LeetCode #684
- **Min Cost to Connect All Points** *(Kruskal's / Prim's MST)* — LeetCode #1584
- **Critical Connections in a Network (Bridges)** *(Tarjan's Algorithm)* — LeetCode #1192

■ Module 10: Dynamic Programming (DP)

10.1 Core Concepts & Theory

- Core Requirements: **Overlapping Subproblems** & **Optimal Substructure**.
- Approaches:
  - **Top-Down (Memoization):** Recursive exploration + Cache lookup.
  - **Bottom-Up (Tabulation):** Iterative DP table population.
  - **Space Optimization:** Reducing state tables from $O(N^2) \to O(N)$ or $O(N) \to O(1)$ by identifying necessary prior states.

10.2 Theoretical Subtopics & Categories

- **1D DP:** Fibonacci sequence pattern, House Robber, Climbing Stairs.
- **2D / Grid DP:** Unique Paths, Minimum Path Sum, Matrix traversals.
- **Knapsack Patterns:**
  - **0/1 Knapsack:** Item inclusion/exclusion once (Subset Sum, Equal Partition).

  - **Unbounded Knapsack:** Infinite supply of items (Coin Change, Rod Cutting).
- **Longest Common Subsequence (LCS):** String matching DP transitions, Edit Distance, Shortest Common Supersequence.
- **Longest Increasing Subsequence (LIS):** *O(N^2)* DP formulation vs *O(N \log N)* Binary Search patience sorting formulation.
- **Interval DP / Matrix Chain Multiplication (MCM):** Subsegment split optimizations.
- **Tree DP:** State transitions across subtrees (Diameter, Max Path Sum).
- **Bitmask DP:** State compression over set elements.

10.3 Must-Do Famous Problems

- **Climbing Stairs** *(1D DP)* — LeetCode #70
- **House Robber I & II** *(1D DP, Circular)* — LeetCode #198 / #213
- **Coin Change I & II** *(Unbounded Knapsack)* — LeetCode #322 / #518
- **Partition Equal Subset Sum** *(0/1 Knapsack)* — LeetCode #416
- **Longest Common Subsequence** *(2D String DP)* — LeetCode #1143
- **Edit Distance** *(2D String DP)* — LeetCode #72
- **Longest Increasing Subsequence** *(LIS, O(N \log N) Binary Search)* — LeetCode #300
- **Word Break** *(DP / Trie)* — LeetCode #139
- **Unique Paths I & II** *(Grid DP)* — LeetCode #62 / #63
- **Maximum Product Subarray** *(1D DP, Min/Max Tracking)* — LeetCode #152
- **Palindrome Partitioning II** *(String DP, Min Cut)* — LeetCode #132
- **Burst Balloons** *(Interval DP / MCM)* — LeetCode #312

■ Module 11: Advanced Data Structures & String Algorithms

11.1 Core Concepts & Theory

- Advanced String Searching and Range Query Structures designed for scalable query optimization.

11.2 Theoretical Subtopics & Patterns

- **Trie (Prefix Tree):** Node structure with child pointers array `children[26]` and `isEndOfWord` flag. *O(L)* insertion/search where *L* is word length.
- **Segment Tree:** Complete binary tree for range queries (Range Sum, Range Min/Max) and point updates in *O(\log N)* time. Build tree in *O(N)*.
- **Binary Indexed Tree (Fenwick Tree / BIT):** Prefix sum range queries and point updates using bitwise low-bit manipulation (`idx & (-idx)`) with *O(N)* space and *O(\log N)* query time.
- **KMP (Knuth-Morris-Pratt) Algorithm:** String pattern searching in *O(N + M)* time using Longest Prefix Suffix (LPS) array.

11.3 Must-Do Famous Problems

- **Implement Trie (Prefix Tree)** *(Trie Design)* — LeetCode #208
- **Design Add and Search Words Data Structure** *(Trie, Backtracking)* — LeetCode #211
- **Maximum XOR of Two Numbers in an Array** *(Bitwise Trie)* — LeetCode #421
- **Range Sum Query - Mutable** *(Segment Tree / Fenwick Tree)* — LeetCode #307
- **Find the Index of the First Occurrence in a String (KMP)** *(KMP Algorithm / LPS)* — LeetCode #28
- **Shortest Palindrome** *(KMP Algorithm / String)* — LeetCode #214